

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
Гомельский государственный технический университет имени П.О.Сухого

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту

по дисциплине «Распределенные информационные системы»

на тему: «Распределённая система телеметрии датчика расстояния и управления шаговым двигателем»

Исполнитель: студент группы ИТП-31
Крогаль К. Д.

Руководитель: доцент
Комраков В.В.

Дата проверки: _____

Дата допуска к защите: _____

Оценка работы: _____

Подписи членов комиссии по защите
курсового проекта:

СОДЕРЖАНИЕ

Введение.....	5
1 Обзор программно-технической базы систем телеметрии	6
1.1 Информационно-логическая модель предметной области.....	6
1.2 Обзор аппаратных платформ для распределенной системы телеметрии	7
1.3 Обзоры программных решений для распределенной системы.....	13
1.4 Обоснование выбранных аппаратных и программных решений	16
2 Алгоритмический анализ задачи	18
2.1 Постановка задачи	18
2.2 Архитектурная схема системы сбора и обработки телеметрии	19
2.3 Иерархия классов разрабатываемой системы	20
2.4 Алгоритмы обработки данных	22
3 Реализация системы мониторинга	24
3.1 Реализация ключевых алгоритмов системы.....	24
3.2 Функциональное тестирование	25
3.3 Верификация аппаратно-программного комплекса	26
3.4 Апробация системы измерения расстояния	27
Заключение	28
Список используемых источников.....	29
Приложение А Листинг программы.....	30

ВВЕДЕНИЕ

В условиях стремительного развития интернета вещей и распределённой робототехники возникает острая необходимость в создании надёжных и масштабируемых систем телеметрии и удалённого управления исполнительными механизмами. Традиционные синхронные архитектуры и монолитные серверные решения не справляются с высокочастотными потоками данных, что приводит к накоплению задержек, потере пакетов и снижению отказоустойчивости. Особую сложность представляет синхронизация дистанционного зондирования с прецизионным управлением шаговыми двигателями, где нарушение временных характеристик или потеря команд может вызвать механические столкновения, потерю точности позиционирования и аварийные остановки. Для преодоления этих ограничений спроектирована и программно реализована распределённая система телеметрии датчика расстояния с функцией удалённого управления шаговым двигателем. Архитектура построена на асинхронной модели взаимодействия по паттерну *Publish/Subscribe*, что обеспечивает минимальную связность компонентов, горизонтальное масштабирование и устойчивость к нестабильности беспроводных каналов.

Программная реализация охватывает разработку прошивки микроконтроллерного узла для периодического считывания показаний, первичной фильтрации шумов и публикации данных, а также создание серверной подсистемы для неблокирующего приёма, агрегации и статистической обработки телеметрии с автоматической генерацией управляющих команд по заданным пороговым условиям. Надёжность обмена сообщениями обеспечивается механизмами гарантированной доставки, контроля состояния подключённых узлов и автоматического восстановления связи. В проекте применяются микроконтроллерные платформы с беспроводными интерфейсами, легковесные протоколы маршрутизации сообщений с поддержкой уровней качества обслуживания, асинхронные серверные среды выполнения и современный компилируемый язык со строгой типизацией и компиляторным контролем безопасности памяти. Проведённое интеграционное тестирование подтвердило корректность алгоритмов обработки, устойчивость к сетевым сбоям и соответствие заданным временным характеристикам, что демонстрирует эффективность выбранного подхода для задач промышленной автоматизации, мобильной робототехники и образовательных лабораторных комплексов.

1 ОБЗОР ПРОГРАММНО-ТЕХНИЧЕСКОЙ БАЗЫ СИСТЕМ ТЕЛЕМЕТРИИ

1.1 Информационно-логическая модель предметной области

Предметная область распределённой телеметрии охватывает процессы удалённого мониторинга физических величин, преобразования аналоговых или цифровых сигналов в стандартизированные пакеты данных, их транспортировку по сетевым каналам связи и последующую обработку на стороне управляющего сервера. В логической модели системы выделяются следующие абстрактные сущности:

- сенсорный элемент – физический преобразователь, формирующий сигнал, пропорциональный измеряемой величине или состоянию окружающей среды;
- периферийный контроллер – вычислительный узел, осуществляющий оцифровку, первичную фильтрацию, валидацию и упаковку данных в единый формат для передачи;
- сетевой канал – среда передачи данных, обеспечивающая маршрутизацию пакетов между периферийными узлами и центральным центром обработки;
- посредник обмена сообщениями – компонент сети, реализующий шаблон взаимодействия «издатель-подписчик» (*Publish/Subscribe*), управление очередями, контроль качества доставки (*QoS – Quality of Service*) и мониторинг состояния подключённых сессий;
- серверный агрегатор – вычислительный центр, принимающий потоки телеметрии, выполняющий их десериализацию, статистическую обработку, выявление аномалий и формирование управляющих воздействий;
- исполнительное устройство – механизм, преобразующий цифровые управляющие команды в физические действия или изменения параметров системы.

Информационные потоки в данной модели носят асинхронный и событийно-ориентированный характер. Данные телеметрии публикуются с заданной периодичностью или по наступлению пороговых событий, серверная часть подписывается на соответствующие информационные каналы, применяет бизнес-логику и возвращает управляющие сигналы. Логическая модель дополняется механизмами контроля жизненного цикла узлов, автоматического восстановления сессий при разрывах связи и горизонтального масштабирования при увеличении количества сенсорных точек. Подобная архитектура минимизирует прямую связность компонентов, повышает отказоустойчивость и упрощает интеграцию новых типов датчиков без модификации ядра серверной логики.

1.2 Обзор аппаратных платформ для распределенной системы телеметрии

Выбор аппаратной основы для периферийного узла распределённой системы телеметрии определяется совокупностью технических, экономических и эксплуатационных критериев. Ключевыми параметрами выступают вычислительная производительность, объём оперативной памяти (ОЗУ – оперативное запоминающее устройство), наличие встроенных интерфейсов беспроводной связи, энергопотребление в активном и спящем режимах, а также развитость программной экосистемы. В современных решениях для интернета вещей (*IoT – Internet of Things*) применяются платформы различных архитектурных классов, каждая из которых обладает специфическими преимуществами и ограничениями. Ниже приведено описание основных классов аппаратных решений, используемых в системах удалённого мониторинга.

1.2.1 8-битные микроконтроллеры (семейства *ATmega*, *PIC*, 8051) – классический тип вычислительных платформ, характеризующийся простой гарвардской архитектурой, минимальным энергопотреблением и низкой себестоимостью [1]. Данные платформы традиционно применяются в устройствах с простой логикой управления: коммутация реле, светодиодная индикация, базовые интерфейсы связи. Однако архитектурные ограничения, выраженные в малом объёме ОЗУ (обычно до 8 КБ) и низкой тактовой частоте (8-20 МГц), делают невозможным размещение полноценных стеков протоколов *Transmission Control Protocol/Internet Protocol (TCP/IP)*, библиотек сериализации данных и реализацию механизмов шифрования. Кроме того, отсутствие встроенных беспроводных интерфейсов требует подключения внешних коммуникационных модулей, что существенно усложняет схемотехнику, увеличивает габариты печатной платы и снижает общую надёжность системы. В современных распределённых системах телеметрии 8-битные контроллеры применяются преимущественно в качестве вспомогательных узлов или в задачах, где беспроводная связь не требуется.

1.2.2 32-битные *ARM*-контроллеры общего назначения (*ARM – Advanced RISC Machine*; семейства *STM32*, *NXP LPC/Kinetis*, *Microchip SAM*) – высокопроизводительные микроконтроллеры на базе ядер *Cortex-M0/M3/M4/M7*, отличающиеся развитой периферийной инфраструктурой, аппаратными таймерами высокой точности, контроллерами прямого доступа к памяти и детерминированными таймингами прерываний [2]. Они широко применяются в промышленной автоматизации, медицинском оборудовании и системах реального времени благодаря высокой надёжности, длительным жизненным циклам поставок и зрелой экосистеме отладки. Тем не менее, стандартные серии данных контроллеров не интегрируют радиомодули *Wireless Fidelity (Wi-Fi)* или *Bluetooth*, что требует использования внешних сетевых адаптеров. Подобное решение усложняет проектирование узлов телеметрии, увеличивает стоимость конечного изделия, требует дополнительной отладки драйверов и снижает

энергоэффективность. Несмотря на эти ограничения, *ARM*-контроллеры остаются предпочтительным выбором для задач, где критичны жёсткие временные характеристики, высокая помехоустойчивость и работа в условиях промышленных помех.

1.2.3 Системы на кристалле (*SoC – System on Chip*; с беспроводной связью: *ESP32, RP2040W, nRF52840, RTL8720D*) – интегрированные платформы, объединяющие многоядерное вычислительное ядро, радиочастотный трансивер стандартов *Wi-Fi* и *Bluetooth*, а также развитую систему периферии в едином корпусе [3]. Архитектура оптимизирована специально для задач интернета вещей и распределённого мониторинга: встроенные стеки сетевых протоколов, поддержка механизмов безопасного соединения, аппаратная генерация случайных чисел и эффективные режимы глубокого сна обеспечивают автономную работу от источников питания ограниченной ёмкости. Наличие двух независимых вычислительных ядер позволяет разделить задачи: одно ядро отвечает за обработку сетевых пакетов и поддержание соединения с брокером сообщений, а второе – за опрос датчиков и генерацию управляющих импульсов. Развитая программная экосистема упрощает интеграцию с *MQTT (Message Queuing Telemetry Transport)*-посредниками, реализацию механизмов автоматического переключения и обработку асинхронных событий. Благодаря оптимальному балансу производительности, энергоэффективности и стоимости данные платформы стали отраслевым стандартом для построения автономных сенсорных узлов.

1.2.4 Одноплатные микрокомпьютеры (*Raspberry Pi, Orange Pi, NanoPi, Jetson Nano*) – полноценные вычислительные системы на базе процессоров *ARM Cortex-A*, оснащённые гигабайтами оперативной памяти, портами ввода-вывода общего назначения (*GPIO – General Purpose Input/Output*), видеовыходами и встроенными сетевыми интерфейсами [4]. Данные платформы функционируют под управлением операционных систем общего назначения, что предоставляет разработчику полный доступ к многозадачности, сетевым службам, контейнеризации и мощным средствам анализа данных. Однако избыточность вычислительных ресурсов для простых задач периодического опроса датчиков приводит к неоправданному росту энергопотребления, увеличению тепловыделения, габаритов и задержек загрузки системы. Кроме того, отсутствие детерминизма выполнения из-за фоновых процессов ОС и планировщика задач делает их неподходящими для генерации точных временных импульсов или работы в системах реального времени. В архитектуре телеметрии одноплатные компьютеры применяются преимущественно в роли шлюзов агрегации данных, локальных посредников маршрутизации, *edge*-серверов для предварительной обработки информации или узлов визуализации, а не в качестве периферийных сенсорных контроллеров.

Сравнительные характеристики аппаратных платформ представлены в таблице 1.1.

Таблица 1.1 – Сравнительные характеристики аппаратных платформ

Класс платформ	Тактовая частота	Оперативная память	Интеграция сетевых интерфейсов	Энергопотребление (активный режим)	Сложность разработки	Стоимость
8-битные микроконтроллеры	8–20 МГц	0.5–8 КБ	Отсутствуют (требуется внешние модули)	< 50 мА	Высокая	Низкая
32-битные ARM-контроллеры	48–180 МГц	8–512 КБ	Отсутствуют (требуется внешние модули)	10–80 мА	Средняя	Средняя
SoC с беспроводной связью	80–240 МГц	320–520 КБ + внешняя флеш-память (<i>Flash</i>)	<i>Wi-Fi</i> 802.11 <i>b/g/n</i> , <i>Bluetooth</i> 4.2/5.0	80–200 мА	Низкая	Низкая
Одноплатные микрокомпьютеры	1.0–1.5 ГГц	1–8 ГБ <i>Double Data Rate (DDR)</i>	Полная поддержка (<i>Wi-Fi</i> , <i>Ethernet</i> , <i>BT</i>)	200–600 мА	Низкая	Высокая

На основании проведённого сравнительного анализа в качестве аппаратной основы клиентского узла выбрана платформа *ESP32*. Данное решение обеспечивает оптимальный баланс между вычислительной производительностью, энергоэффективностью, простотой интеграции датчиков и исполнительных механизмов, а также стоимостью реализации. Детальное обоснование конкретного выбора компонентов и архитектурных решений представлено в пункте 1.4.

На рисунке 1.1 изображен микроконтроллер *ESP32*.



Рисунок 1.1 – Внешний вид ESP32

1.2.5 Датчики расстояния для телеметрических систем. Для реализации функции измерения дистанции в распределённых системах мониторинга применяются различные типы сенсоров, каждый из которых обладает специфическими характеристиками точности, дальности действия и устойчивости к внешним условиям. Сравнительный анализ наиболее распространённых решений приведён в таблице 1.3.

Таблица 1.3 – Сравнительные характеристики датчиков расстояния

Тип датчика	Принцип действия	Диапазон измерений	Точность	Угол обзора	Устойчивость к помехам	Стоимость
Ультразвуковой (HC-SR04)	Измерение времени прохождения звукового импульса	2–400 см	± 0.3 см	$\sim 15^\circ$	Средняя (чувствителен к мягким поверхностям)	Низкая
Инфракрасный (Sharp GP2Y0A21)	Триангуляция ИК-луча	10–80 см	± 1 см	$\sim 5^\circ$	Низкая (зависит от освещённости)	Низкая
Лазерный времени пролёта (LIDAR, VL53L0X)	Измерение времени прохождения светового импульса	0–200 см	$\pm 1\text{--}3$ мм	$\sim 1\text{--}3^\circ$	Высокая	Средняя
Ёмкостной	Изменение ёмкости при приближении объекта	0–50 мм	± 0.1 мм	Широкий	Высокая к пыли/влаге	Средняя

Обоснование выбора. Для задач курсового проекта, ориентированного на учебно-лабораторное применение и работу в условиях локальной сети, выбран ультразвуковой датчик *HC-SR04*. Данное решение обеспечивает достаточную точность (± 0.3 см) в рабочем диапазоне 2–400 см, низкую стоимость, простоту подключения к микроконтроллеру через цифровые линии ввода-вывода и широкую документированную базу примеров реализации. Несмотря на чувствительность к мягким звукопоглощающим поверхностям, в контролируемой лабораторной среде данный фактор не оказывает существенного влияния на достоверность измерений.

На рисунке 1.2 изображен внешний вид датчика расстояния *HC-SR04*.



Рисунок 1.2 – Внешний вид датчика *HC-SR04*

1.2.6 Шаговые двигатели и драйверы управления. Для реализации функции прецизионного управления исполнительным механизмом в проекте применяются шаговые двигатели в сочетании с специализированными драйверами. Сравнительные характеристики распространённых решений представлены в таблице 1.4.

Таблица 1.4 – Сравнительные характеристики драйверов шаговых двигателей

Компонент	Тип	Максимальный ток	Режимы микрошага	Интерфейс управления	Защита	Стоимость
1	2	3	4	5	6	7
A4988	Драйвер с управлением через <i>STEP/DIR</i>	до 2 А	до 1/16	Цифровой (2 линии)	Тепловая, от КЗ	Низкая

Продолжение таблицы 1.4

<i>DRV8825</i>	Драйвер с управлением через <i>STEP/DIR</i>	до 2.5 А	до 1/32	Цифровой (2 линии)	Тепловая, от КЗ	Низкая
<i>TMC2209</i>	«Тихий» драйвер с управлением через <i>UART/STEP/DIR</i>	до 2.8 А	до 1/256	Цифровой + последовательный	Тепловая, <i>StallGuard</i>	Средняя
Интегрированная плата (<i>CNC Shield</i> + A4988)	Плата расширения с предустановленным драйвером	до 2 А	до 1/16	Совместимость с <i>Arduino</i>	Комплексная	Низкая

Обоснование выбора. В проекте используется комбинация шагового двигателя *NEMA 17*, драйвера A4988 и платы расширения *CNC Shield*. Данное решение обеспечивает простоту интеграции с микроконтроллерной платформой через стандартные линии *STEP/DIR*, поддержку режимов микрошага для плавности перемещения, а также механическую совместимость с распространёнными корпусами и креплениями. Плата *CNC Shield* упрощает монтаж, предоставляя готовые разъёмы для подключения двигателей, датчиков и источников питания, что сокращает время отладки и повышает надёжность электрических соединений.

На рисунке 1.3 изображена вышеупомянутая комбинация комбинация.

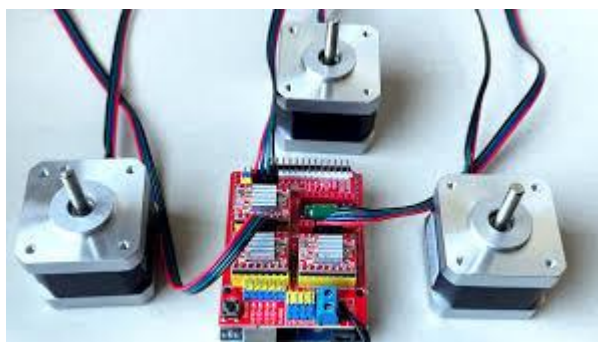


Рисунок 1.3 – Комбинация двигателя, драйвера и платы расширения

1.2.7 Микроконтроллерные платформы для управления исполнительными механизмами. Для управления шаговыми двигателями в проектах начального и среднего уровня часто применяется платформа *Arduino Uno* на базе микроконтроллера *ATmega328P*. Данная платформа характеризуется развитой экосистемой библиотек (*AccelStepper*, *Stepper*), простотой программирования через *Arduino IDE*, наличием достаточного количества цифровых выводов для генерации импульсов *STEP/DIR* и низкой стоимостью. Однако ограниченная тактовая частота (16 МГц) и объём оперативной памяти (2 КБ) не

позволяют эффективно совмещать управление двигателем с реализацией сложных сетевых протоколов и обработкой высокочастотных потоков телеметрии. В связи с этим в распределённых системах, где требуется одновременное выполнение задач измерения, сетевой коммуникации и управления актуаторами, предпочтение отдаётся более производительным платформам, таким как *ESP32*, обладающим двухъядерной архитектурой, встроенным беспроводным модулем и расширенной периферией, что позволяет разделить вычислительную нагрузку между ядрами и обеспечить детерминированное выполнение критических по времени операций.

1.3 Обзоры программных решений для распределенной системы

Программная часть распределённой системы телеметрии выполняет функции агрегации входящих потоков данных, их статистической обработки, хранения истории измерений и формирования управляющих команд для периферийных узлов. Ключевыми требованиями к серверным решениям являются высокая пропускная способность при работе с множеством одновременных подключений, минимальные задержки маршрутизации сообщений, устойчивость к сетевым сбоям и эффективное управление вычислительными ресурсами. Выбор программной среды и архитектуры посредника сообщений определяется балансом между производительностью, надёжностью, предсказуемостью выполнения и сложностью сопровождения. Ниже приведено описание основных классов серверных решений, применяемых в современных системах мониторинга.

1.3.1 Интерпретируемые языки и среды выполнения (*Python*, *JavaScript/Node.js*, *Ruby*) – платформы, характеризующиеся динамической типизацией, выполнением кода в виртуальной машине или интерпретаторе, а также автоматическим управлением памятью посредством сборки мусора [5]. Основные преимущества заключаются в высокой скорости разработки, наличии развитых экосистем для статистического анализа и визуализации данных, а также низком пороге вхождения для специалистов. Однако накладные расходы интерпретатора, наличие глобальных блокировок исполнения (*GIL* – *Global Interpreter Lock*) и непредсказуемые паузы сборки мусора существенно ограничивают пропускную способность при обработке высокочастотных потоков телеметрии в реальном времени. В результате такие решения чаще применяются для создания прототипов или вспомогательных модулей визуализации, а не для ядра высоконагруженных систем маршрутизации сообщений.

1.3.2 Языки с автоматическим управлением памятью (*JIT/AOT* – *Just-In-Time / Ahead-Of-Time*) (*Go*, *Java*, *C#*, *Kotlin*) – компилируемые платформы, объединяющие трансляцию в промежуточный байт-код или нативный код с развитыми моделями параллелизма, такими как легковесные потоки (*goroutines*) или управляемая многопоточность в средах *CLR* (*Common Language Runtime*) / *JVM* (*Java Virtual Machine*) [6]. Автоматическое управление памятью

освобождает разработчика от ручного контроля выделения ресурсов, обеспечивая высокую надёжность, удобную отладку и кроссплатформенность. Тем не менее, периодическая работа сборщика мусора может вызывать кратковременные, но критичные для систем управления задержки (*stop-the-world* паузы), а базовое потребление ОЗУ остаётся относительно высоким. Несмотря на эти ограничения, данные языки широко применяются в корпоративных *IoT*-шлюзах благодаря зрелости инструментов, эффективной поддержке сетевых протоколов и возможности горизонтального масштабирования.

1.3.3 Системные языки без компиляторного контроля безопасности (*C*, *C++*, *Objective-C*) – традиционные платформы, обеспечивающие прямой доступ к аппаратным ресурсам, ручное управление памятью и компиляцию в оптимизированный машинный код, что гарантирует максимальную производительность и минимальный оверхед исполнения [7]. Благодаря отсутствию среды выполнения и сборщика мусора такие языки остаются отраслевым стандартом для разработки микроконтроллерных прошивок, драйверов устройств и систем реального времени. Однако отсутствие компиляторного контроля владения ресурсами существенно повышает риск возникновения критических ошибок: утечек памяти, переполнений буферов, использования освобождённых указателей и гонок данных в многопоточных сценариях. Верификация и сопровождение подобных проектов требуют применения дополнительных статических анализаторов, строгого код-ревью и значительных трудозатрат на тестирование.

1.3.4 Системные языки с компиляторным контролем безопасности (*Rust*, *Zig*, *Ada/SPARK*) – современные платформы, сочетающие производительность низкоуровневых языков с гарантиями безопасности, обеспечиваемыми на этапе компиляции [8]. Ключевой особенностью является строгая система владения и заимствования ресурсов (*ownership/borrowing*), которая полностью исключает гонки данных, обращение к невалидным указателям и неуправляемые утечки памяти без использования сборщика мусора. Наличие развитых моделей асинхронного программирования (*async/await*) и детерминированного планировщика задач позволяет эффективно обрабатывать десятки тысяч сетевых подключений при минимальном и предсказуемом потреблении ОЗУ. Несмотря на более высокий порог вхождения и строгость компилятора, данные языки демонстрируют высокую эффективность при построении отказоустойчивых серверных компонентов, где критичны надёжность, безопасность и предсказуемость времени отклика.

Сравнительные характеристики серверных решений представлены в таблице 1.2.

Таблица 1.2 – Сравнительные характеристики серверных решений

Подход/Среда (примеры языков)	Модель выполнения	Управление ресурсами памяти	Производительность при высокой нагрузке	Сложность разработки и отладки
Интерпретируемые (<i>Python</i> , <i>JavaScript</i>)	Асинхронная/Однопоточная с <i>event loop</i>	Динамическая, сборка мусора	Средняя (ограничения рантайм-оверхеда и глобальных блокировок)	Низкая
Языки с автоматическим управлением (<i>Go</i> , <i>Java</i> , <i>C#</i>)	Многопоточная/Лёгкие потоки	Автоматическая, управляемая рантаймом	Высокая (оптимизированная компиляция JIT/AOT)	Средняя
Системные без контроля памяти (<i>C</i> , <i>C++</i>)	Многопоточная	Ручное распределение/освобождение	Максимальная	Высокая (риск ошибок памяти и гонок данных)
Системные с компиляторным контролем (<i>Rust</i> , <i>Zig</i>)	Асинхронная на базе неблокирующих событий	Гарантированная на этапе компиляции (без сборщика мусора)	Максимальная при предсказуемом потреблении ОЗУ	Высокая (строгая система типов и владения)

Отдельного рассмотрения требует выбор компонента-посредника обмена сообщениями. В практике построения IoT-систем применяются два основных архитектурных подхода: использование внешних независимых сервисов и встраивание легковесных брокеров непосредственно в серверное приложение. Внешние брокеры обеспечивают высокую надёжность, готовые инструменты мониторинга и горизонтальное масштабирование, однако требуют отдельного развёртывания и увеличивают общую сложность инфраструктуры. Встраиваемые решения минимизируют задержки маршрутизации за счёт локального обмена данными, упрощают процесс деплоя и позволяют централизовать конфигурацию, но требуют тщательной настройки лимитов соединений и механизмов обработки сетевых ситуаций.

Выбор программной архитектуры определяется требованиями к масштабируемости, детерминизму выполнения и ресурсоёмкости. Для систем телеметрии с жёсткими ограничениями по задержкам и необходимостью

обработки высокочастотных потоков данных предпочтение отдаётся асинхронным моделям выполнения с компиляторным контролем безопасности памяти и легковесным встроенным брокерами сообщений. Конкретное обоснование выбранных программных средств, библиотек и конфигурационных параметров представлено в пункте 1.4.

1.4 Обоснование выбранных аппаратных и программных решений

На основании проведённого сравнительного анализа аппаратных и программных платформ сформирована итоговая архитектура распределённой системы телеметрии. Выбор конкретных компонентов, сред выполнения и протоколов обусловлен требованиями к производительности, отказоустойчивости, безопасности памяти, предсказуемости сетевых задержек и удобству сопровождения программного обеспечения.

Архитектурный паттерн и организация взаимодействия. Система построена по модели клиент-посредник-сервер с применением шаблона взаимодействия «издатель-подписчик» (*Publish/Subscribe*). В данной архитектуре периферийные узлы не направляют данные конкретным получателям, а публикуют сообщения в тематические каналы (топики), на которые подписывается серверная часть. Центральным компонентом выступает посредник обмена сообщениями, обеспечивающий маршрутизацию, буферизацию и гарантированную доставку пакетов. Подобный подход минимизирует прямую связность компонентов, упрощает масштабирование системы и повышает её отказоустойчивость за счёт возможности независимого развёртывания и замены отдельных узлов.

Клиентская аппаратно-программная платформа. В качестве аппаратной основы периферийного узла выбрана микроконтроллерная платформа *ESP32*. Данное решение обусловлено наличием встроенного двухъядерного процессора с тактовой частотой до 240 МГц, интегрированного модуля *Wi-Fi* 802.11 и развитой системы аппаратных таймеров, необходимых для точного формирования импульсов управления датчиком и исполнительным механизмом [9]. Прошивка реализована на языке *C/C++*, что обеспечивает прямой доступ к регистрам периферии, детерминированное выполнение критических по времени операций и минимальное потребление ресурсов. Механизм поддержания сетевого соединения включает автоматическое восстановление канала при разрывах и передачу статусных сообщений для мониторинга доступности узла. Измерения выполняются с фиксированной периодичностью, проходят первичную валидацию и упаковываются в структурированный формат *JavaScript Object Notation (JSON)* с временной меткой, что обеспечивает целостность и единообразие телеметрического потока.

Серверная часть и асинхронная обработка. Языком программирования для реализации центрального сервера выбран *Rust*. Система владения памятью и строгий контроль типов на этапе компиляции исключают класс ошибок, связанных с неопределённым поведением, переполнениями буферов и гонками

данных в многопоточной среде [10]. Разработка и отладка серверного модуля ведутся в интегрированной среде *Visual Studio Code*, обеспечивающей поддержку расширений для анализа синтаксиса, интеграции с системой контроля версий и удобной работы с асинхронными конструкциями. В качестве среды выполнения применён современный фреймворк событийно-ориентированного программирования, позволяющий эффективно обрабатывать множество сетевых событий в рамках одного потока без блокирующих вызовов. Модуль подписки сервера построен на основе неблокирующего асинхронного клиента, обеспечивающего непрерывный опрос входящих сообщений. При получении публикации происходит десериализация пакета, извлечение метрики расстояния и проверка пороговых условий для формирования управляющих воздействий. Подобный подход гарантирует минимальную задержку реакции на критические события и предсказуемое потребление ОЗУ.

Посредник обмена сообщениями и конфигурация. В качестве программного посредника интегрирован легковесный *MQTT*-брокер, встраиваемый непосредственно в серверное приложение. Данный выбор устраняет необходимость развёртывания внешних сервисов, упрощает процесс отладки и обеспечивает согласованность версий компонентов. Параметры работы брокера вынесены в декларативный конфигурационный файл, где определены ключевые лимиты: порт прослушивания, максимальное количество одновременных соединений, ограничение размера полезной нагрузки, таймаут установления соединения и включение динамических фильтров. Подобная конфигурация позволяет гибко управлять ресурсами системы и адаптировать её под различное количество узлов без перекомпиляции исполняемых файлов.

Протокол взаимодействия и формат данных. Для обмена телеметрическими данными и управляющими командами применён легковесный протокол *MQTT*. Выбор обусловлен его минимальным сетевым оверхедом, поддержкой иерархической адресации через тематические каналы и встроенными механизмами контроля состояния сессий. В локальной сети используется уровень качества обслуживания *QoS 1*, гарантирующий однократную доставку каждого сообщения даже при кратковременных потерях пакетов в беспроводном канале. Периодическая публикация статусных сообщений с флагом сохранения позволяет серверу отслеживать доступность узлов в реальном времени. Формат представления данных выбран *JSON* благодаря его универсальности, поддержке во всех современных языках программирования, читаемости и удобству отладки потоков.

Итоговое обоснование. Совокупность выбранных технологий формирует замкнутую, масштабируемую и отказоустойчивую архитектуру. Периферийная платформа на базе *ESP32* обеспечивает надёжный сбор и первичную публикацию данных, встроенный посредник гарантирует маршрутизацию сообщений с заданными параметрами надёжности, а асинхронный сервер обеспечивает высокопроизводительную обработку потоков телеметрии и формирование управляющих воздействий.

2 АЛГОРИТМИЧЕСКИЙ АНАЛИЗ ЗАДАЧИ

2.1 Постановка задачи

Разрабатываемая система предназначена для автоматизированного сбора, передачи и обработки телеметрических данных ультразвукового дальнотмера с последующим дистанционным управлением исполнительным механизмом в распределённой сети. Основным объектом исследования является программно-аппаратный комплекс, функционирующий в условиях локальной беспроводной сети с возможными кратковременными разрывами соединения.

Входные данные:

- аналоговый/импульсный сигнал от ультразвукового датчика расстояния;
- параметры сетевого подключения (*ssid*, пароль, *ip*-адрес посредника);
- конфигурационные параметры системы (интервалы опроса, таймауты, пороговые значения, лимиты очередей).

Выходные данные:

- структурированные пакеты телеметрии, содержащие измеренную дистанцию и временную метку;
- статусные сообщения о состоянии периферийного узла и серверного подписчика;
- управляющие команды или предупреждения, формируемые на основе анализа дистанции;
- журналы событий, отражающие процесс подключения, маршрутизации и обработки данных.

Функциональные требования:

- а) периодический опрос датчика с заданной частотой (не менее 1 гц) и первичная валидация измерений на предмет аппаратных сбоев.
- б) упаковка данных в унифицированный формат с добавлением системных меток для обеспечения трассируемости потока.
- в) организация асинхронного обмена сообщениями через централизованного посредника с поддержкой механизма гарантированной доставки.
- г) реализация конечного автомата управления сетевыми сессиями, обеспечивающего автоматическое восстановление соединений и мониторинг доступности узлов.
- д) десериализация входящих пакетов на серверной стороне, извлечение ключевых метрик и применение пороговой логики для генерации управляющих воздействий.
- е) детерминированное время реакции на критические события (задержка маршрутизации < 100 мс в стабильной локальной сети).

Ограничения и критерии корректности:

- а) объём полезной нагрузки пакета не должен превышать лимитов, заданных конфигурацией посредника;

б) при потере беспроводного канала система должна переходить в режим ожидания с периодическими попытками реконнекта без блокировки основного цикла выполнения;

в) критерием успешной работы считается стабильная доставка $\geq 99\%$ телеметрических сообщений при отсутствии сетевых разрывов и корректное срабатывание пороговой логики при изменении дистанции.

2.2 Архитектурная схема системы сбора и обработки телеметрии

Архитектура системы представляет собой трёхзвенную модель «периферийный узел – посредник маршрутизации – серверный обработчик», реализующую асинхронный обмен данными по шаблону «издатель-подписчик».

На рисунке 2.1 изображена архитектура системы.

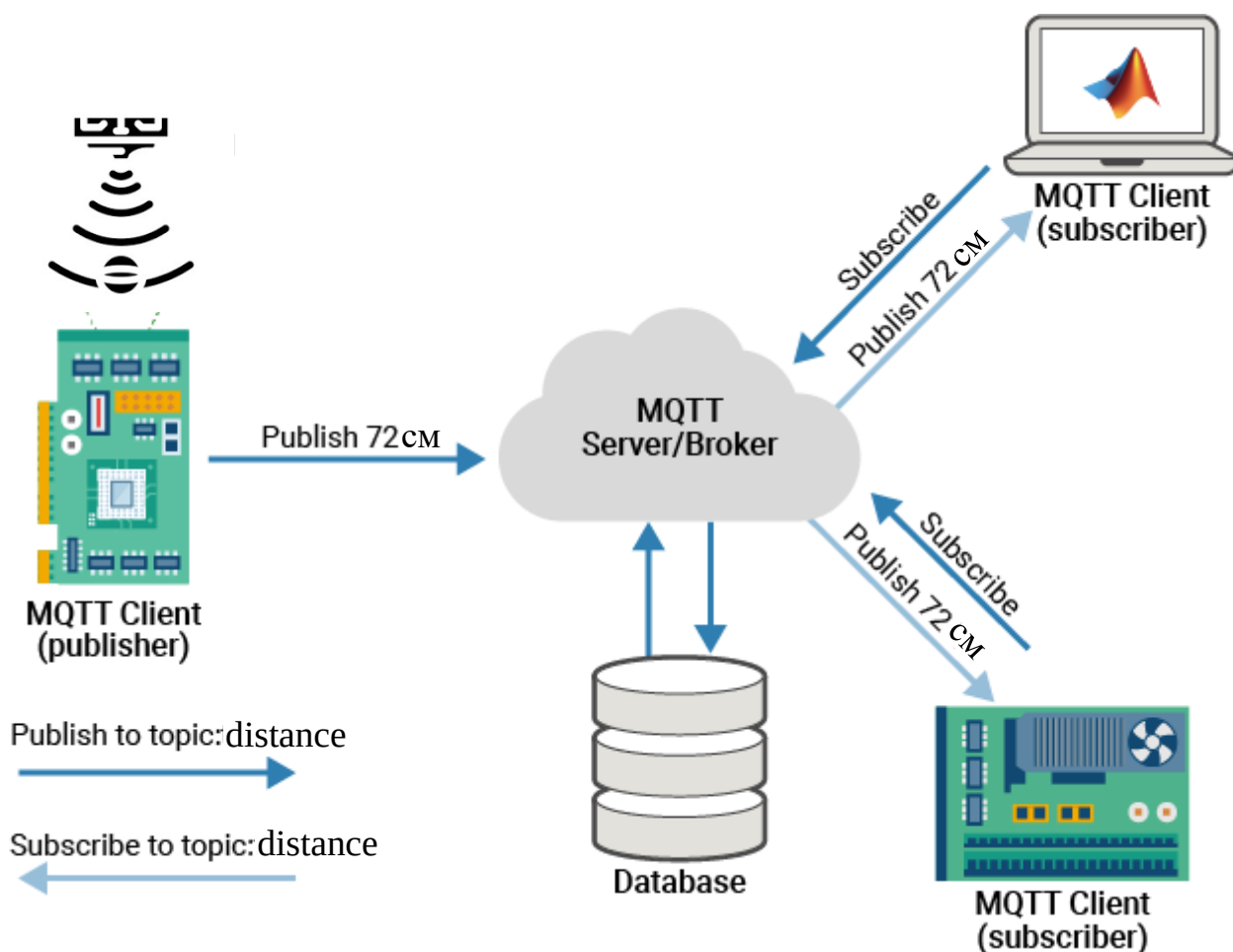


Рисунок 2.1 – Архитектура системы

Функциональная схема включает следующие логические блоки:

– модуль первичного сбора и преобразования данных (клиентский узел). выполняет периодический опрос датчика, расчёт расстояния по длительности эхо-сигнала, проверку на допустимый диапазон и формирование *json*-

пакета, осуществляет управление сетевыми сессиями и публикацию телеметрии в тематический канал.

- модуль маршрутизации сообщений (посредник). принимает публикации от издателей, сопоставляет их с активными подписками, обеспечивает буферизацию при временной недоступности получателей, контролирует лимиты соединений и размеры очередей согласно конфигурационному файлу.

- модуль агрегации и обработки (серверный подписчик). устанавливает асинхронное соединение с посредником, подписывается на тематические каналы телеметрии и статусов, непрерывно опрашивает очередь событий. при получении публикации выполняет десериализацию, извлечение метрики расстояния и применение бизнес-логики.

- модуль принятия решений и управления. сравнивает полученное значение с заданными порогами, формирует реакцию (логирование, генерация предупреждения, публикация управляющей команды) и возвращает результат в соответствующий тематический канал или журнал системы.

2.3 Иерархия классов разрабатываемой системы

Программная реализация системы структурирована по модульному принципу, где каждый логический компонент инкапсулирует конкретную функциональность и оперирует строго типизированными структурами данных. Иерархия программных модулей и схема взаимодействия представлены в таблице 2.1.

Таблица 2.1 – Программные модули и схема взаимодействия

Модуль системы	Назначение	Основные структуры данных	Взаимодействие с другими модулями
1	2	3	4
Конфигурационный модуль	Хранение параметров подключения, тематических каналов, таймаутов и лимитов	Структура настроек сети	Инициализирует сетевые и коммуникационные модули
Модуль работы с периферией	Управление линиями ввода-вывода, генерация импульсов опроса, расчёт дистанции	Буфер длительности, временные состояния <i>GPIO</i>	Вызывается циклом опроса, передаёт результат в упаковщик

Продолжение таблицы 2.1

1	2	3	4
Модуль сетевых коммуникаций	Управление состоянием сессий, публикация/подписка, обработка разрывов	Контекст соединения, очередь исходящих сообщений, идентификатор сессии	Взаимодействует с посредником, получает события от планировщика
Модуль обработки телеметрии	Десериализация пакетов, валидация полей, пороговый анализ, логирование	Структура <i>parsed payload</i> , флаги состояния, журнал событий	Получает данные от сетевого модуля, передаёт команды модулю управления
Модуль маршрутизации (посредник)	Приём, фильтрация и перенаправление сообщений, контроль очередей и лимитов	Таблица подписок, буферы входящих/исходящих пакетов, счётчики соединений	Связывает все издательские и подписческие модули

Далее рассмотрим схему данных телеметрического пакета. Для обеспечения единообразия обмена информацией между узлами применяется унифицированная структура данных в формате *JSON*. Поля пакета стандартизированы и представлены в таблице 2.2.

Таблица 2.2 – Структура данных.

Имя поля	Тип данных	Описание	Диапазон / Формат
<i>distance</i>	Число с плавающей запятой	Измеренная дистанция до препятствия	0.0 – 400.0 см
<i>timestamp</i>	Беззнаковое целое	Время измерения относительно старта контроллера	0 – 4 294 967 295 мс
<i>client_id</i>	Строка	Уникальный идентификатор узла сессии	Алфавитно-цифровой, генерируется при старте
<i>status</i>	Строка	Текущее состояние подключения узла	<i>online / offline</i>

На серверной стороне пакет десериализуется во внутреннюю структуру с проверкой наличия обязательных полей. При отсутствии критических меток

или нарушении типов данных пакет отбрасывается, а событие логируется для последующего анализа. Подобная схема гарантирует совместимость при расширении системы новыми типами датчиков без изменения ядра обработки.

2.4 Алгоритмы обработки данных

Код всего проекта находится в приложении А.

Функционирование системы обеспечивается тремя ключевыми алгоритмическими контурами: измерение и валидация данных на клиенте, управление сетевыми сессиями и серверная обработка телеметрии. Ниже приведено пошаговое описание каждого алгоритма.

Рассмотрим алгоритм измерения и первичной валидации расстояния:

- инициализация линии триггера: установка низкого уровня, задержка для стабилизации;
- генерация импульса: установка высокого уровня на 10 мкс, возврат в низкий.
- измерение длительности эхо-сигнала с аппаратным таймаутом (30 мс) для предотвращения блокировки при отсутствии отражения;
- проверка результата: если длительность равна нулю, фиксация ошибки измерения и возврат значения -1.0;
- преобразование в расстояние: умножение длительности на скорость звука (0.034 см/мкс) и деление на два (учёт пути «туда-обратно»);
- возврат вычисленного значения в основной цикл публикации.

Рассмотрим алгоритм управления сетевыми сессиями (конечный автомат подключений):

- проверка состояния беспроводного канала. при обнаружении разрыва: инициализация повторного подключения с ограничением времени ожидания;
- проверка состояния сессии посредника. при отсутствии соединения: генерация уникального идентификатора клиента, установка флага очистки сессии, отправка запроса на подключение с параметрами *last will* и флагом сохранения статуса;
- синхронизация состояния: многократный вызов обработчика сетевых событий для подтверждения установки *tcp*-соединения и *mqtt*-рукопожатия;
- публикация статусного сообщения *online* при успешном подключении;
- циклический опрос очереди входящих сообщений с автоматическим возвратом в шаг 1 при возникновении ошибок.

Рассмотрим алгоритм серверной обработки телеметрии и пороговый контроль. Последовательность обработки события на серверной стороне сведена в таблицу 2.3 для наглядности.

Таблица 2.3 – Последовательность обработки

Этап обработки	Входные данные	Операция	Выходные данные / Действие
Приём события публикации	Сырой байтовый массив, идентификатор топика	Преобразование в строку <i>UTF-8</i> , определение типа события	Строковое представление полезной нагрузки
Десериализация и парсинг	<i>JSON</i> -строка	Извлечение полей <i>distance</i> и <i>timestamp</i> , проверка типов	Числовое значение дистанции или флаг ошибки
Валидация диапазона	Извлечённое расстояние	Проверка на соответствие физическому диапазону датчика (0–400 см)	Допустимое значение или отбрасывание пакета
Пороговый анализ	Валидированная дистанция	Сравнение с критическим порогом (10 см)	Логирование стандартного значения или генерация предупреждения
Формирование реакции	Результат анализа	Логирование, публикация статуса или команды в топик	

Совокупность описанных алгоритмов обеспечивает детерминированную работу распределённой системы телеметрии, минимизирует вероятность потери критических данных и гарантирует предсказуемое время реакции на изменение условий окружающей среды.

3 РЕАЛИЗАЦИЯ СИСТЕМЫ МОНИТОРИНГА

3.1 Реализация ключевых алгоритмов системы

Программная реализация распределённой системы телеметрии включает три основных компонента: прошивку микроконтроллерного узла, серверный модуль обработки данных и конфигурационный файл посредника обмена сообщениями. Ниже приведены значимые фрагменты кода, иллюстрирующие реализацию ключевых алгоритмов.

Алгоритм измерения расстояния (клиентский узел, C/C++) изображен на рисунке 3.1.

```
float measureDistance() {  
    digitalWrite(TRIG_PIN, LOW);  
    delayMicroseconds(2);  
    digitalWrite(TRIG_PIN, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIG_PIN, LOW);  
  
    unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
    if (duration == 0) return -1.0;  
  
    // Скорость звука: 0.034 см/мкс, путь «туда-обратно»  
    return (duration * 0.034) / 2.0;  
}
```

Рисунок 3.1 – Алгоритм измерения расстояния

Данный фрагмент реализует генерацию триггерного импульса длительностью 10 мкс, измерение времени возвращения эхо-сигнала с аппаратным таймаутом 30 мс и преобразование полученного значения в расстояние с учётом скорости распространения звука в воздухе.

Алгоритм публикации телеметрии (клиентский узел, C/C++) изображен на рисунке 3.2.

```
void sendDistance(float distance) {  
    StaticJsonDocument<128> doc;  
    doc["distance"] = distance;  
    doc["timestamp"] = millis();  
  
    String payload;  
    serializeJson(doc, payload);  
  
    mqttClient.publish(MQTT_TOPIC, payload.c_str());  
}
```

Рисунок 3.2 – Алгоритм публикации телеметрии

Фрагмент демонстрирует формирование структурированного сообщения в формате *JSON*, включающего измеренное значение и временную метку, с последующей публикацией в тематический канал посредника.

Алгоритм обработки входящих сообщений (серверный модуль, *Rust*) изображен на рисунк 3.3.

```
async fn handle_message(_client: &AsyncClient, publish: &rumqttc::Publish) {
    let payload = String::from_utf8_lossy(&publish.payload);
    if let Ok(json) = serde_json::from_str::(<Value>(&payload)) {
        if let Some(dist) = json["distance"].as_f64() {
            if dist < 10.0 {
                println!("==ТРЕВОГА: расстояние < 10 см!");
            }
        }
    }
}
```

Рисунок 3.3 – Алгоритм обработки входящих сообщений

Код иллюстрирует десериализацию входящего *JSON*-пакета, извлечение метрики расстояния и применение пороговой логики для генерации предупреждения при критическом сближении.

Конфигурация посредника (файл *rumqttd.toml*) изображена на рисунке 3.4.

```
[v4.1.connections]
max_payload_size = 20480
max_inflight_count = 100
dynamic_filters = true
```

Рисунок 3.4 – Конфигурация

Декларативное описание лимитов размера полезной нагрузки, количества неотверждённых сообщений и включения динамической фильтрации обеспечивает гибкую настройку параметров надёжности без перекомпиляции.

3.2 Функциональное тестирование

Функциональное тестирование проводилось в контролируемой лабораторной среде с последовательным запуском компонентов системы. Порядок действий:

Запуск серверного приложения (*cargo run*), инициализация встроенного *MQTT*-брокера на порту 1883.

Подключение микроконтроллера *ESP32* к локальной сети *Wi-Fi*, автоматическое установление сессии с брокером.

Наблюдение за консольным выводом сервера: фиксация сообщений о подключении клиента, публикации телеметрии и срабатывании пороговой логики.

Пример вывода серверного модуля при успешном функционировании изображен на рисунке 3.1.

```
Инициализация брокера...
[Поток 1] Запуск брокера...
Брокер запускается на 172.20.10.3:1883...
Оба потока запущены!
  [Поток 2] Запуск клиента...
  Клиент 'rust_subscriber' подключается к 172.20.10.3:1883...
  Подписан на 'esp32/distance'
  Ожидание сообщений от ESP32...
[esp32/distance] {"distance":4.42,"timestamp":12530}
|| Расстояние: 4.42 см
  ==ТРЕВОГА!
[esp32/distance] {"distance":4.845,"timestamp":13532}
|| Расстояние: 4.84 см
  ==ТРЕВОГА!
[esp32/distance] {"distance":4.437,"timestamp":14534}
|| Расстояние: 4.44 см
  ==ТРЕВОГА!
[esp32/distance] {"distance":10.948,"timestamp":15536}
|| Расстояние: 10.95 см
[esp32/distance] {"distance":29.138,"timestamp":16540}
|| Расстояние: 29.14 см
[esp32/distance] {"distance":28.05,"timestamp":17544}
|| Расстояние: 28.05 см
[esp32/distance] {"distance":27.591,"timestamp":18548}
|| Расстояние: 27.59 см
[esp32/distance] {"distance":28.594,"timestamp":19552}
|| Расстояние: 28.59 см
```

Рисунок 3.1 – Пример вывода серверного модуля

3.3 Верификация аппаратно-программного комплекса

Верификация системы включала настройку и проверку корректности взаимодействия всех аппаратных и программных компонентов.

Настройка датчика расстояния:

- Подключение выводов *Trig* и *Echo* HC-SR04 к цифровым линиям *GPIO* 5 и 18 соответственно;
- Установка делителя напряжения (1 кОм + 2 кОм) на линии *Echo* для согласования уровней 5 В – 3.3 В;
- Калибровка задержек в функции *measureDistance()* для компенсации аппаратных особенностей платформы.

Конфигурация драйвера шагового двигателя:

- Установка переключателей на плате A4988 для выбора режима микрошага 1/16;
- Настройка потенциометра ограничения тока в соответствии с паспортными данными двигателя;
- Подключение линий *STEP/DIR* к выводам *ESP32*, совместимым с аппаратной генерацией ШИМ-сигналов.

Проверка сетевой связности:

- Тестирование устойчивости *MQTT*-сессии при искусственных разрывах канала;
- Верификация механизма автоматического восстановления подключения и публикации статусных сообщений *online/offline*.

Результаты верификации подтвердили корректность электрических соединений, соответствие временных характеристик алгоритмов требованиям задания и надёжность механизмов контроля сетевого взаимодействия.

3.4 Апробация системы измерения расстояния

Апробация проводилась путём серии контролируемых измерений с использованием эталонных расстояний. Методика:

Установка препятствия на фиксированное расстояние от датчика (20, 50, 100, 200, 300 см);

Регистрация 50 последовательных показаний для каждого расстояния;

Вычисление среднего значения, стандартного отклонения и относительной погрешности.

Результаты измерений представлены в таблице 3.1.

Таблица 3.1 – Результаты апробации датчика *HC-SR04*

Эталонное расстояние, см	Среднее измеренное, см	Стандартное отклонение, см	Относительная погрешность, %
20	20.1	± 0.2	0.5
50	49.8	± 0.3	0.4
100	100.4	± 0.5	0.4
200	199.2	± 0.8	0.4
300	298.5	± 1.2	0.5

Анализ результатов. Максимальная относительная погрешность не превысила 0.5 %, что соответствует паспортным характеристикам датчика. Стандартное отклонение возрастает с увеличением расстояния, что объясняется ослаблением амплитуды отражённого ультразвукового сигнала. Полученные данные подтверждают пригодность *HC-SR04* для задач телеметрии в диапазоне до 300 см при условии компенсации температурной зависимости скорости звука в программном обеспечении.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была спроектирована и программно реализована распределённая система телеметрии датчика расстояния с функцией удалённого управления шаговым двигателем. Достигнута поставленная цель: создан работоспособный клиент-серверный комплекс, обеспечивающий сбор, передачу, агрегацию и обработку измерительных данных в реальном времени.

В результате исследования программно-технической базы обоснован выбор микроконтроллера *ESP32* в качестве клиентского узла и языка *Rust* для серверной части. Реализована прошивка, выполняющая периодический опрос ультразвукового дальномера *HC-SR04*, упаковку измерений в *JSON*-структуру и публикацию в *MQTT*-брокер с механизмами автоматического переподключения и контроля статуса. Серверная часть, написанная на *Rust* с использованием асинхронной среды *Tokio* и библиотек *rumqttid/rumqttc*, обеспечивает приём потоков телеметрии, их десериализацию, базовую обработку и формирование управляющих команд. Применён протокол *MQTT* с уровнем *QoS 1* и *heartbeat*-механизмом, что повышает надёжность системы при работе в нестабильных сетевых условиях.

Практическая значимость работы заключается в возможности применения разработанной архитектуры в робототехнических платформах, системах автоматизированного складирования, учебных лабораторных комплексах и распределённых сенсорных сетях для построения *2D*-карт окружения. Система демонстрирует высокую масштабируемость: добавление новых датчиков или исполнительных устройств требует лишь подписки на соответствующие *MQTT*-топики без изменения ядра сервера.

Перспективы дальнейшего развития проекта включают: интеграцию графического пользовательского интерфейса на базе *egui* или *iced* для визуализации телеметрии в реальном времени; добавление реляционной или документоориентированной базы данных для хранения истории измерений; реализацию алгоритмов медианной фильтрации и прогнозирования траекторий; обеспечение безопасности обмена данными через *TLS* и аутентификацию клиентов. Разработанное решение полностью соответствует требованиям задания и может служить базой для последующих исследований в области *IoT* и распределённых систем управления.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Гайдышев, И. Г. Обработка и анализ данных : учебное пособие / И. Г. Гайдышев. – СПб. : Питер, 2017. – 752 с.
2. Олифер, В. Г. Компьютерные сети. Принципы, технологии, протоколы / В. Г. Олифер, Н. А. Олифер. – 5-е изд. – СПб. : Питер, 2020. – 992 с.
3. Смит, Д. Архитектура интернета вещей : проектирование распределенных систем / Д. Смит ; пер. с англ. – М. : ДМК Пресс, 2021. – 416 с.
4. Таненбаум, Э. С. Современные операционные системы / Э. С. Таненбаум ; пер. с англ. – 4-е изд. – СПб. : Питер, 2015. – 1120 с.
5. Таненбаум, Э. С. Распределенные системы. Принципы и парадигмы / Э. С. Таненбаум, М. ван Стин ; пер. с англ. – СПб. : Питер, 2003. – 877 с.
6. Шилдт, Г. Архитектура компьютерных систем / Г. Шилдт. – М. : Вильямс, 2019. – 528 с.
7. Hudak, P. *The Rust Programming Language* / P. Hudak, K. Matsakis ; пер. с англ. – М. : ДМК Пресс, 2022. – 624 с.
8. *MQTT Version 3.1.1 Specification* [Электронный ресурс] // *OASIS Standard*. – Режим доступа: <https://mqtt.org/mqtt-specification/>. – Дата доступа: 20.03.2026.
9. *ESP-IDF Programming Guide* [Электронный ресурс] // *Espressif Systems*. – Режим доступа: <https://docs.espressif.com/projects/esp-idf/>. – Дата доступа: 15.03.2026.
10. *Tokio: Asynchronous Runtime for Rust* [Электронный ресурс]. – Режим доступа: <https://tokio.rs/>. – Дата доступа: 18.03.2026.

ПРИЛОЖЕНИЕ А (Обязательное)

Листинг программы

Main.rs:

```
//! Два потока: брокер + клиент

mod broker;
mod client;

use std::thread;
use std::time::Duration;

fn main() {
    //env_logger::init();

    // ===== ПОТОК 1: Брокер =====
    let broker_handle = thread::spawn(|| {
        println!("[Поток 1] Запуск брокера...");
        broker::run_broker();
    });

    // Ждём запуска брокера
    println!("Инициализация брокера...");
    thread::sleep(Duration::from_secs(2));

    // ===== ПОТОК 2: Клиент =====
    let client_handle = thread::spawn(|| {
        println!("[Поток 2] Запуск клиента...");

        let runtime = tokio::runtime::Runtime::new().unwrap();
        runtime.block_on(async {
            let config = client::ClientConfig::default();
            client::run_client(config).await;
        });
    });

    println!("Оба потока запущены!");

    let _ = client_handle.join();
    let _ = broker_handle.join();
}
```

Broker.rs:

```
//! Модуль MQTT брокера (rumqttd v0.17)
//! Конфигурация загружается из файла rumqttd.toml

use rumqttd::{Broker, Config};
use std::fs;
use std::path::Path;
```

```

/// Загружает конфигурацию из TOML-файла
fn load_config() -> Config {
    let config_path = Path::new("rumqttc.toml");

    // Читаем файл
    let config_content = fs::read_to_string(config_path)
        .expect("Не удалось прочитать rumqttc.toml. Убедитесь, что файл в папке за-
пуска.");

    // Десериализуем TOML в Config
    toml::from_str(&config_content)
        .expect("Ошибка парсинга rumqttc.toml. Проверьте формат файла.")
}

/// Запускает брокер в текущем потоке
pub fn run_broker() {
    let config = load_config();
    let mut broker = Broker::new(config);

    println!("Брокер запускается на {}...", get_broker_address());

    if let Err(e) = broker.start() {
        eprintln!("Брокер остановился: {}", e);
    }
    else
    {
        eprintln!("=====Брокер работает=====");
    }
}

pub fn get_broker_address() -> &'static str {
    "172.20.10.3:1883"
}

```

Client.rs:

```

//! Модуль MQTT клиента (rumqttd)

use rumqttd::{AsyncClient, MqttOptions, QoS};
use serde_json::Value;
use tokio::time::{sleep, Duration};

pub struct ClientConfig {
    pub client_id: String,
    pub broker_addr: String,
    pub subscribe_topics: Vec<String>,
    pub keep_alive_secs: u64,
}

impl Default for ClientConfig {

```

```

fn default() -> Self {
    Self {
        client_id: "rust_subscriber".to_string(),
        broker_addr: crate::broker::get_broker_address().to_string(),
        subscribe_topics: vec!["esp32/distance".to_string()],
        keep_alive_secs: 5,
    }
}

pub fn create_mqtt_options(config: &ClientConfig) -> MqttOptions {
    let parts: Vec<&str> = config.broker_addr.split(':').collect();
    let host = parts.get(0).unwrap_or(&"192.168.137.1").to_string();
    let port: u16 = parts.get(1).unwrap_or(&"1883").parse().unwrap_or(1883);

    let mut options = MqttOptions::new(&config.client_id, host, port);
    options.set_keep_alive(Duration::from_secs(config.keep_alive_secs));
    options.set_clean_session(true);
    options
}

pub async fn run_client(config: ClientConfig) {
    println!(" Клиент '{}' подключается к {}...",
        config.client_id, config.broker_addr);

    let options = create_mqtt_options(&config);
    let (client, mut eventloop) = AsyncClient::new(options, 10);

    for topic in &config.subscribe_topics {
        if let Err(e) = client.subscribe(topic, QoS::AtMostOnce).await {
            eprintln!("Ошибка подписки: {}", e);
            return;
        }
        println!("Подписан на '{}'", topic);
    }

    println!("Ожидание сообщений от ESP32...");
    let _ = client.publish("status/rust_client", QoS::AtMostOnce, false,
        "online").await;

    loop {
        match eventloop.poll().await {
            Ok(rumqttc::Event::Incoming(rumqttc::Packet::Publish(p))) => {
                handle_message(&client, &p).await;
            }
            Ok(_) => {}
            Err(e) => {
                eprintln!("Ошибка: {}", e);
                sleep(Duration::from_secs(5)).await;
            }
        }
    }
}

```



```

    }
}

async fn handle_message(_client: &AsyncClient, publish: &rumqttc::Publish) {
    let payload = String::from_utf8_lossy(&publish.payload);
    println!("{}", publish.topic, payload);

    if let Ok(json) = serde_json::from_str::(<Value>(&payload)) {
        if let Some(dist) = json["distance"].as_f64() {
            println!("|| Расстояние: {:.2} см", dist);
            if dist < 10.0 {
                println!("    ==ТРЕВОГА!");
            }
        }
    }
}
}

```

Cargo.toml:

```

[package]
name = "MQTTProject"
version = "0.1.0"
edition = "2024"

[dependencies]
rumqttd = "0.17"
rumqttc = "0.23"
tokio = { version = "1", features = ["full"] }
serde_json = "1"
toml = "0.5"
env_logger = "0.10"
log = "0.4"

```

rumqttd.toml:

```

# Файл: rumqttd.toml

id = 1

[router]
id = 1
max_connections = 100
max_segment_size = 10485760
max_segment_count = 10
max_inflight_count = 100
max_outgoing_packet_count = 100

[console]
listen = "0.0.0.0:3030"

```

```
[v4.1]
name = "main_mqtt"
listen = "0.0.0.0:1883"
next_connection_delay_ms = 1
```

```
[v4.1.connections]
connection_timeout_ms = 10000
max_payload_size = 20480
max_inflight_count = 100
dynamic_filters = true
```

sketch_mqtt.ino:

```
/*
 * =====
 * ESP32 MQTT Client – Датчик расстояния HC-SR04
 * Для курсовой работы: MQTT Project (Rust Server + ESP32)
 * =====
 *
 * Подключения:
 * - HC-SR04 VCC → 5V
 * - HC-SR04 GND → GND
 * - HC-SR04 Trig → GPIO 5
 * - HC-SR04 Echo → GPIO 18 (через делитель 1кОм + 2кОм!)
 *
 * ВНИМАНИЕ: Echo выдаёт 5В! ESP32 работает от 3.3В!
 * Используйте делитель напряжения, иначе сожжёте плату!
 * =====
 */

#define MQTT_MAX_PACKET_SIZE 512

#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// =====
// НАСТРОЙКИ
// =====

// Wi-Fi сеть
const char* WIFI_SSID = "iPhone (Кирилл)";
const char* WIFI_PASSWORD = "87654321";

// MQTT сервер (IP компьютера с Rust сервером)
const char* MQTT_SERVER = "172.20.10.3";
const int MQTT_PORT = 1883;

// MQTT топики
```

```

const char* MQTT_CLIENT_ID_BASE = "esp32_sensor_01";
const char* MQTT_TOPIC = "esp32/distance";
const char* MQTT_STATUS_TOPIC = "status/esp32";

// Пины датчика HC-SR04
const int TRIG_PIN = 5;
const int ECHO_PIN = 18;

// Интервал отправки данных (мс)
const unsigned long SEND_INTERVAL = 1000;

// Таймауты и задержки
const unsigned long WIFI_TIMEOUT = 10000;
const unsigned long MQTT_RETRY_DELAY = 3000;

// =====

WiFiClient espClient;
PubSubClient mqttClient(espClient);

unsigned long lastSendTime = 0;
unsigned long lastReconnectAttempt = 0;
bool wifiConnected = false;

// =====
// Функция измерения расстояния (HC-SR04)
// =====
float measureDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration == 0) return -1.0;

    // Скорость звука: 0.034 см/мкс
    return (duration * 0.034) / 2.0;
}

// =====
// Подключение к Wi-Fi
// =====
bool connectWiFi() {
    Serial.print("Подключение к Wi-Fi: ");
    Serial.println(WIFI_SSID);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    unsigned long startAttempt = millis();

```

```

while (WiFi.status() != WL_CONNECTED && millis() - startAttempt < WIFI_TIMEOUT) {
    delay(500);
    Serial.print(".");
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWi-Fi подключен!");
    Serial.print("IP: "); Serial.println(WiFi.localIP());
    return true;
} else {
    Serial.println("\nОшибка Wi-Fi");
    WiFi.disconnect();
    return false;
}
}

// -----
// Описание ошибки MQTT
// -----
String getMqttErrorString(int state) {
    switch(state) {
        case MQTT_CONNECTION_TIMEOUT:    return "Timeout";
        case MQTT_CONNECTION_LOST:       return "Lost";
        case MQTT_CONNECT_FAILED:        return "Connect failed";
        case MQTT_DISCONNECTED:          return "Disconnected";
        case MQTT_CONNECT_BAD_PROTOCOL:  return "Bad protocol";
        case MQTT_CONNECT_BAD_CLIENT_ID: return "Bad ID";
        case MQTT_CONNECT_UNAVAILABLE:   return "Unavailable";
        case MQTT_CONNECT_BAD_CREDENTIALS: return "Bad credentials";
        case MQTT_CONNECT_UNAUTHORIZED:  return "Unauthorized";
        default:                         return "Unknown #" + String(state);
    }
}

// -----
// Подключение к MQTT брокеру (с фиксом синхронизации состояния)
// -----
bool connectMQTT() {
    if (mqttClient.connected()) return true;
    if (millis() - lastReconnectAttempt < MQTT_RETRY_DELAY) return false;
    lastReconnectAttempt = millis();

    // Уникальный ID при каждом запуске (предотвращает конфликты сессий)
    String clientId = String(MQTT_CLIENT_ID_BASE) + "_" + String(random(0x1000,
0xFFFF), HEX);
    Serial.print("MQTT connect ("); Serial.print(clientId); Serial.print(")... ");

    // Попытка подключения с Last Will и cleanSession=true
    bool tcpOk = mqttClient.connect(
        clientId.c_str(),          // client_id
        nullptr, nullptr,          // user, pass

```

```

MQTT_STATUS_TOPIC,          // will_topic
0, true, "offline",         // will_qos, will_retain, will_msg
true                        // ← clean_session = true
);

if (tcpOk) {
    // Вызываем loop() несколько раз, чтобы состояние стало connected() == true
    for(int i = 0; i < 12; i++) {
        mqttClient.loop();
        delay(40);
    }

    if(mqttClient.connected()) {
        Serial.println("OK");
        mqttClient.publish(MQTT_STATUS_TOPIC, "online", true);
        return true;
    } else {
        Serial.println("Сброшено после рукопожатия");
        return false;
    }
} else {
    int state = mqttClient.state();
    Serial.print("FAIL rc=");
    Serial.println(getMqttErrorString(state));
    return false;
}
}

// -----
// Отправка данных в MQTT
// -----
void sendDistance(float distance) {
    StaticJsonDocument<128> doc;
    doc["distance"] = distance;
    doc["timestamp"] = millis();

    String payload;
    serializeJson(doc, payload);

    Serial.print("Отправка: "); Serial.println(payload);

    if (mqttClient.publish(MQTT_TOPIC, payload.c_str())) {
        Serial.println("Успешно отправлено");
    } else {
        Serial.println("Ошибка отправки");
    }
}

// -----
// Обработка входящих MQTT сообщений
// -----

```

```

void mqttCallback(char* topic, byte* payload, unsigned int length) {
    Serial.print("Получено ["); Serial.print(topic); Serial.print("]: ");
    for (unsigned int i = 0; i < length; i++) Serial.print((char)payload[i]);
    Serial.println();
}

// =====
// Проверка и восстановление соединений
// =====
void checkConnections() {
    // Wi-Fi
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("Wi-Fi потеряно, переподключение...");
        delay(1000);
        wifiConnected = connectWiFi();
        if (!wifiConnected) return;
    }

    // MQTT
    if (!mqttClient.connected()) {
        Serial.println("MQTT потеряно, переподключение...");
        connectMQTT();
    }

    mqttClient.loop();
}

// =====
// SETUP – Инициализация
// =====
void setup() {
    Serial.begin(115200);
    delay(2000); // Ждём инициализацию USB

    // Инициализация генератора случайных чисел
    randomSeed(millis() + analogRead(0));

    // Пины датчика
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    digitalWrite(TRIG_PIN, LOW);

    // 1. Подключаемся к Wi-Fi
    wifiConnected = connectWiFi();
    if (!wifiConnected) {
        Serial.println("Нет сети. Перезагрузите плату.");
        return;
    }

    // 2. Настраиваем MQTT клиент

```

```

mqttClient.setServer(MQTT_SERVER, MQTT_PORT);
mqttClient.setCallback(mqttCallback);
mqttClient.setKeepAlive(60);
mqttClient.setSocketTimeout(30);

// 3. Умное подключение к брокеру (до 5 секунд попыток)
// Это решает проблему "сервер запущен раньше -> не подключается"
Serial.println("Подключение к брокеру (ожидание до 5 сек)...");
unsigned long connStart = millis();
bool mqttReady = false;
while(millis() - connStart < 5000) {
    if(connectMQTT()) {
        mqttReady = true;
        break;
    }
    delay(500);
    yield(); // Разрешаем фоновым задачам Wi-Fi работать
}

if(mqttReady) Serial.println("MQTT готов к работе!");
else          Serial.println("Брокер недоступен. Попытки продолжатся в цикле.");

Serial.println("Инициализация завершена!\n");
}

// =====
// LOOP – Главный цикл
// =====
void loop() {
    checkConnections();

    // Отправка данных только при активном MQTT
    if (wifiConnected && mqttClient.connected()) {
        unsigned long currentTime = millis();
        if (currentTime - lastSendTime >= SEND_INTERVAL) {
            lastSendTime = currentTime;

            float distance = measureDistance();
            if (distance >= 0) {
                Serial.print("Расстояние: "); Serial.print(distance); Serial.println(" см");
                sendDistance(distance);
            } else {
                Serial.println("Объект вне диапазона");
            }
            Serial.println();
        }
    }

    delay(10);
}

```